C#

Les bases du langage, expliquées simplement

1. C'est quoi le C# ?

Le **C#** (« C sharp ») est un langage de programmation créé par **Microsoft** en 2000. Il fait partie de la plateforme **.NET**. C'est un langage **orienté objet**, **compilé** et **typé** (chaque variable a un type précis). Il sert à créer des applications Windows, des sites web (ASP.NET), des API, des applications mobiles et des jeux vidéo (avec le moteur **Unity**).

> **Pour programmer en C# :** installer **Visual Studio** (ou **VS Code**) et le **SDK .NET**. Pour débuter, on crée un projet « Application console » : le programme s'exécute dans une fenêtre de texte.

2. Le premier programme

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Bonjour !");
    }
}
```

Ligne	Rôle
`using System;`	Permet d'utiliser les outils de base de .NET (comme `Console`)
`class Program`	Une classe : le contenant du code
`static void Main()`	Le **point d'entrée** : le programme commence ici
`Console.WriteLine(...)`	Affiche du texte dans la console, puis passe à la ligne

Dans les versions récentes de .NET, on peut écrire directement `Console.WriteLine("Bonjour !");` sans la classe ni le `Main`. Les exemples suivants sont à placer **dans** la méthode `Main`. Chaque instruction se termine par un `;`.

Les **commentaires** servent à expliquer le code et sont ignorés :

```
// commentaire sur une ligne
/* commentaire
   sur plusieurs lignes */
```

3. Les variables et les types

Une **variable** est une case mémoire qui porte un nom et contient une valeur. On indique son **type** puis son nom :

Type	Contient	Exemple
`int`	Un nombre entier	`int age = 19;`
`double`	Un nombre à virgule	`double prix = 12.5;`
`string`	Du texte	`string nom = "Léa";`
`char`	Un seul caractère	`char lettre = 'A';`
`bool`	Vrai ou faux	`bool majeur = true;`

Avec `var`, C# devine le type tout seul : `var age = 19;` (c'est un `int`).

4. Afficher et lire

```
Console.Write("Ton prénom : ");          // affiche sans passer à la ligne
string prenom = Console.ReadLine();       // lit ce que l'utilisateur tape

Console.WriteLine("Salut " + prenom + " !");   // concaténation
Console.WriteLine($"Salut {prenom} !");        // texte interpolé (plus simple)

Console.Write("Ton âge : ");
int age = int.Parse(Console.ReadLine());  // convertit le texte en nombre
```

`Console.ReadLine()` renvoie toujours du **texte**. Pour obtenir un nombre, on le convertit avec `int.Parse(...)`.

5. Les opérateurs

Famille	Opérateurs	Exemple
Calcul	`+ - * / %`	`7 % 2` donne `1` (reste de la division)
Comparaison	`== != < > <= >=`	`age >= 18`
Logique	`&&` (et), `||` (ou), `!` (non)	`age >= 18 && age < 65`
Affectation	`= += -= ++ --`	`x += 1;` ou `x++;`

> Attention : `=` **donne une valeur** à une variable, `==` **compare** deux valeurs.

6. Les conditions

```
if (age >= 18)
{
    Console.WriteLine("Majeur");
}
else if (age >= 16)
{
    Console.WriteLine("Presque majeur");
}
else
{
    Console.WriteLine("Mineur");
}
```

7. Les boucles

```
// for : quand on connaît le nombre de tours
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine("Tour " + i);
}

// while : tant que la condition est vraie
int compteur = 0;
while (compteur < 3)
{
    Console.WriteLine(compteur);
    compteur++;
}

// foreach : parcourir une collection (voir plus bas)
foreach (int note in notes)
{
    Console.WriteLine(note);
}
```

8. Tableaux et listes

```
int[] notes = { 12, 15, 9 };            // tableau : taille fixe
Console.WriteLine(notes[0]);            // affiche 12 (on compte à partir de 0)
Console.WriteLine(notes.Length);        // 3 éléments

List<string> prenoms = new List<string>();   // liste : taille variable
prenoms.Add("Léa");
prenoms.Add("Hugo");
Console.WriteLine(prenoms.Count);       // 2
```

Pour utiliser `List`, il faut ajouter `using System.Collections.Generic;` en haut du fichier (déjà inclus automatiquement dans les projets récents).

9. Les méthodes (fonctions)

Une **méthode** est un bloc de code réutilisable, avec un nom, des paramètres et une valeur de retour :

```
static int Additionner(int a, int b)
{
    return a + b;
}

static void DireBonjour(string nom)     // void = ne renvoie rien
{
    Console.WriteLine("Bonjour " + nom);
}

// Utilisation (dans Main)
int somme = Additionner(3, 4);          // somme vaut 7
DireBonjour("Léa");
```

10. Les classes et les objets

Une **classe** est un plan de construction ; un **objet** est une chose créée à partir de ce plan. Une classe regroupe des **données** (propriétés) et des **actions** (méthodes) :

```
class Personne
{
    public string Nom { get; set; }
    public int Age { get; set; }

    public void Presenter()
    {
        Console.WriteLine($"Je m'appelle {Nom} et j'ai {Age} ans.");
    }
}

// Utilisation (dans Main)
Personne p = new Personne();
p.Nom = "Léa";
p.Age = 19;
p.Presenter();     // Je m'appelle Léa et j'ai 19 ans.
```

11. À retenir

• Le C# est un langage **orienté objet** de Microsoft, utilisé avec **.NET**.

• Un programme console démarre dans la méthode `Main`. Chaque instruction finit par `;`.

• Types de base : `int`, `double`, `string`, `char`, `bool`.

• `if / else` pour décider ; `for`, `while`, `foreach` pour répéter.

• Un **tableau** a une taille fixe, une **List** peut grandir.

• Une **méthode** regroupe du code réutilisable ; une **classe** décrit des objets avec leurs propriétés et méthodes.